CSE 420: Compiler Design

Assignment 3: SDT Rule Combination for Three Address Code Generation

Submission Deadline: Wednesday, Sep 3 (In submission box at 4G06 or in Class)

We have investigated many SDT rules in class. Some of these rules construct the abstract syntax tree, some process variable declarations and populate info in symbol tables, some handle code generations for expressions, regular variable assignments, and array accesses. In the book, all these SDT rules are written separately so that it is easier for the students to understand how they work in isolation. However, when you develop a compiler, you have to combine all these rules with the production rules in your bison file. In this assignment, your task is to do exactly that. That is, write a single attribute grammar that:

1. You will assume that the SDT for constructing an abstract syntax tree (AST) from an input program during the syntax analysis is already available. You do not have to write the rules for tree construction.
2. You start by writing the production and SDT rules for variable declarations, arithmetic expressions, assignment statements, while loops, and if-else branching in a single attribute grammar.
3. You then construct the abstract syntax tree (AST) for a sample program you choose that has the following things:
    a. Declaration of at least three global variables among which at least one is a 2d array.
    b. Some assignments of complex arithmetic expression result to some variables.
    c. A while loop having an if-else inside it or vice versa.
    d. Reading/writing of elements of some 2d array inside the loop.
4. Then populates a symbol table with proper type, width, and offset attributes for variables by traversing the AST using the SDT rules you wrote.
5. Then generates three address codes for entire AST using your SDT rules.

The syllabus document shared at the beginning of the semester mentions where in the book the above topics are discussed and isolated attribute grammars are given.

HINT: you can keep a global flag variable. You write the SDT rules in a manner so that type info processing and symbol table population happens in a tree traversal when the flag is zero. Then set the flag to 1 to do the go-to label setup for code generation using another traversal. Then set the flag to 2 and do the final traversal for code generation.

Don't forget that your submission must be a handwritten hardcopy.

Bonus Assignment Part: You can earn 5 extra bonus points that will compensate your earlier assignments and quizzes if you add the following additional components to your submission.

1. (1 points) support variable declarations within loops and if-else branches.
2. (2 point) support **for loops** in the attribute grammar.
3. (2 points) support function/procedure calls in the attribute grammar.

Late Submission Policy: No late submission will be accepted for this assignment.